

Contents

1	Introduction to Python	4
2	Roles in Analytics	4
3	Types of Analytics	4
4	Jupyter Notebook	4
4.1	Jupyter Notebook State	4
4.1.1	Edit State	4
4.1.2	Select State	4
4.2	Jupyter Notebook Modes	5
5	Comments	5
5.1	Single-Line Comment	5
5.2	Multi-Line String	5
6	Variable	5
7	Built-in Date Types	5
7.1	Checking Data Types	5
7.2	Implicit Type Conversion	6
8	Print	6
9	Operators	6
9.1	Arithmetic Operators	6
9.2	Arithmetic Assignment Operators	6
9.3	Logical Operators	6
9.4	Comparison Operators	6
9.5	Questions	6
10	Conditional Statements	7
10.1	Type 1: if	7
10.2	Type 2: if-else	7
10.3	Type 3: if-elif-else	7
10.4	Type 4: Ternary Conditional Expression	7
10.5	Question	7
11	Keywords	8
12	Input()	8
13	int()	8
14	Loops	8
15	Data Structures	9
15.1	Tuples:	9
15.1.1	Tuple Indexing:	9
15.1.2	Tuple Slicing:	9
15.2	List:	10
15.2.1	Update Operations	10
15.2.2	Deletion Operations	10
15.2.3	Miscellaneous Operations	11
15.3	Dictionary(key-value pair)	11

16 Sets	12
16.1 Set Operations	12
16.1.1 Union of Two Sets	12
16.1.2 Intersection of Two Sets	12
16.1.3 Difference Operation	12
16.1.4 Symmetric Difference	12
17 List Comprehension	12
18 User-Defined Functions(UDF)	12
19 Variable-Length Positional Arguments (*args)	13
20 Variable-Length Keyboard Arguments(**kwargs)	13
21 Lambda Functions	14
22 Object-Oriented Programming	14
23 Class	14
24 Object	14
25 Constructor	14
26 Instance Variables	14
27 Methods	14
28 Getter Methods	15
29 Return Values	15
30 Setter Method	15
31 Creating an Object	15
32 OOP Principles	15
33 Dunder Methods	15
34 dir()	16
35 Installing a Python Library	16
35.1 pip	16
35.2 pip freeze	16
35.3 Package:	16
36 NumPy	17
36.1 Importing NumPy	17
36.2 Creating a NumPy Array	17
36.3 Array Properties and Attributes	17
36.4 Controlling the Data Type	17
36.5 Creating Arrays	18
36.6 Creating Sequences with np.arange()	18
36.7 Creating Evenly Spaced Values with np.linspace()	18
36.8 Generating Random Values with np.random.random()	18
36.9 Generating Random Integers with np.random.randint()	19
36.10Transposing an Array	19
36.11Flattening an Array	19
36.12NumPy Array Slicing and Dicing	19
36.13Reshaping NumPy Arrays	19

36.14	Boolean Filtering	20
36.15	Conditional Selection with np.where()	20
37	Pandas	20
37.1	Series:	20
37.1.1	Properties of a Series	21
37.1.2	Series Attributes	21
37.1.3	loc Indexer:	21
37.1.4	iloc Indexer:	21
37.2	DataFrame:	21
37.2.1	DataFrame Attributes:	22
38	Data Analysis Workflow	22
38.1	Requirement gathering	22
38.2	Data Import	22
38.3	Understanding the data	22
38.4	Data prep—Data Cleaning—Data wrangling—Data munging	22
39	Data Preparation	23
39.1	Structure-based changes	23
39.2	Data-Based Changes	23

1 Introduction to Python

Python is a high-level interpreted, open-source programming language that is widely used for software development, automation, data analysis, artificial intelligence, machine learning, web development, scientific computing, and many other domains.

2 Roles in Analytics

According to the uploaded material, common analytics role include:

- Data Engineer
- Data Analyst/ Business Analyst
- Data Scientist/ Decision Scientist
- Machine Learning/ Artificial Intelligence Engineer
- MLOps (Machine Learning Operations)

3 Types of Analytics

- Descriptive Analytics
- Diagnostic Analytics
- Inferential Analytics
- Predictive Analytics
- Prescriptive Analytics
- Cognitive Analytics

4 Jupyter Notebook

Jupyter Notebook is an interactive development environment that allows users to combine executable Python code, formatted text, mathematical equations, tables, and visualizations in a single document.

4.1 Jupyter Notebook State

Every notebook cell has two states.

4.1.1 Edit State

Used for editing the contents of a cell.

Common shortcuts:

- Ctrl+Enter: Execute current cell and remain in the same cell.
- Shift+Enter: Execute current cell and move to the next cell.

4.1.2 Select State

Used for notebook management.

Important shortcuts:

- ESC+C: Copy cell
- ESC+X: Cut cell
- ESC+V: Paste cell

- ESC+A: Insert cell above
- ESC+B: Insert cell below
- ESC+DD: Delete current cell

4.2 Jupyter Notebook Modes

Three notebook modes are introduced.

- **Code Mode:** Used for writing executable Python code.
Shortcut: ESC+Y
- **RawNBConvert Mode:** Behaves like plain text without formatting.
Shortcut: ESC+R
- **Markdown Mode:** Used to create documentation.
Shortcut: ESC+M

5 Comments

Comments are not executed by the Python interpreter. Primary uses include:

- Documentation
- Recording observations.

5.1 Single-Line Comment

Begins with the symbol #

5.2 Multi-Line String

Python does not have true multi-line comments.

Instead, triple-quoted strings are commonly used for documentation (docstrings).

The uploaded code is reproduced exactly below.

```
# This is the comment
""" This is
multi-line string"""
```

6 Variable

A variable acts as a container that stores a value in memory.

7 Built-in Data Types

Data Type	Description	Conversion Function
int	Whole numbers	int()
float	Decimal numbers	float()
str	Sequence of characters	str()
bool	Logical values	bool()

7.1 Checking Data Types

type(a)-Returns the type of the object.

7.2 Implicit Type Conversion

```
a=20.2
b=10
c=a+b # Implicit conversion of b to float type
type(c)-float
```

8 Print

The function print() prints its argument but returns None.

9 Operators

9.1 Arithmetic Operators

- +
- -
- *
- /
- %
- //

9.2 Arithmetic Assignment Operators

- +=
- -=
- *=
- /=

9.3 Logical Operators

- and
- or
- not

9.4 Comparison Operators

- >
- <
- >=
- <=
- ==
- !=
- in

9.5 Questions

- Identify if the given number is positive (true) or negative (false).

10 Conditional Statements

Conditional Statements allow a program to make decisions based on one or more logical conditions. They control the execution flow by executing blocks of code depending on whether a condition evaluates or True or False.

Types of Conditional Statements

10.1 Type 1: if

if condition:

... statement

The block executes only when the condition is True.

10.2 Type 2: if-else

if condition:

... statement

else:

... statement

Exactly one block executes.

10.3 Type 3: if-elif-else

if condition:

... statement

elif condition:

... statement

...

else:

... statement

Python evaluates the conditions sequentially until one becomes True.

10.4 Type 4: Ternary Conditional Expression

A ternary expression performs conditional evaluation in a single line.

value_if_true if conditions else value_if_false

10.5 Question

- Identify if the given number is even or odd.
- Identify if the given number is
positive even,
negative even
positive odd and
negative odd number
- Given a value, identify if it's numeric or non-numeric type.
- Take a number as an input, and annotate:
'foo' if the number is divisible by 3
'buzz' if the number is divisible by 5
'foobuzz' if the number is divisible by 3 and 5
print the number a such if not divisible by either 3 or 5.
- Identify if a given number is positive or negative with ternary.
- Identify if a given number is positive or negative or 0 with ternary.
- Identify if the given number is even or odd with ternary.

11 Keywords

- try- Contains statements that may raise an exception.
- except- Executes when an exception occurs.
- finally- Executes regardless of whether an exception occurred.

12 Input()

Syntax: `input(prompt)`

Returns user input as a string.

13 int()

Convert a string into an integer.

Syntax: `int(value)`

14 Loops

Loops are control flow statements that allow a block of code to execute repeatedly. They are one of the most important concepts in Python because they automate repetitive tasks and reduce manual coding. Python provides two primary looping constructs:

- for Loop (Range-based Loop)
Syntax:
for variable in iterable:
... statements
- while Loop (Entry Controlled Loop) Syntax:
cnt=0
while condition:
... statements ... cnt+=1 or cnt-=1
- **Important Notes**
 - A while loop can easily become an infinite loop if the stopping condition is not updated correctly.
 - A for loop is generally used when the number of iterations is known.
 - A while loop is suitable when the number of iterations is unknown.

Problems

- Write the table of 5 (for loop)
- Write the table of 5 in inverse manner (for loop)
- Write the table of 5 (while loop)
- write the table of 5 in inverse manner (while loop)
- From a given set of number starting from 1 to 100, identify the sum of even and odd numbers.
- Identify the number of digits in a given number.
- Identify the sum of digits in a given number.

15 Data Structures

Data Structures are specialized containers used to store and organize multiple values efficiently. Try enable efficient **CRUD**(Create, Read, Update, Delete) operations and are fundamental in Python programming, data analytics, and software development.

Python provides several built-in data structures:

15.1 Tuples:

A tuple is an ordered, immutable collection of objects.

Once a tuple is created, its elements cannot be modified.

Syntax: `t1=(1,2,3)`

Characteristics

- Ordered collection.
- Immutable.
- Allows duplicate values.
- Can store heterogeneous data.
- Supports indexing and slicing.
- Faster than lists for read-only data.

Problems

- From a given tuple identify all the numeric values and print out the final sum.

```
t1=(1,2,3)
```

```
t2=(3,4,5.6,'abc',True, False, t1)
```

15.1.1 Tuple Indexing:

A tuple supports zero-based indexing.

15.1.2 Tuple Slicing:

The notebook introduces slicing using the following syntax.

Slicing/Dicing in tuple

Syntax: `[[start]:[end]:[step]]`

- start- starting index
- end- ending index(excluded)
- step- increment/decrement

Rules:

- The starting index is included.
- The ending index is excluded.
- Default step is 1.
- A negative step traverses the tuple in reverse order.

Example:

- `t2[:3]` Output- (3,4,5.6)
- `t2[4:6]` Output- (True, False)

Problems

- Fetch alternative numbers start from 1
t3=tuple(range(1,10))
- Fetch only even numbers from t3
- Find common elements in the given tuples.
t_x = (1,2,3,4,5)
t_y = (4,5,6,7)

15.2 List:

A list is an ordered, mutable collection that can store elements of different data types.

A Python list stores multiple values inside square brackets.

Syntax:

```
variable_name = [item1, item2, item3]
```

Characteristics

- Ordered
- Mutable
- Duplicates Allowed
- Indexed
- Heterogenous Data
- Resizable

Python provides the build-in function dir() to display all available attributes and methods.

The notebook categorizes the operations as follows.

15.2.1 Update Operations

- **append() Method:** The append() method adds exactly one element to the end of a list.
Syntax: list_name.append(element)
Time Complexity: O(1)
- **extend() Method:** The extend() method inserts multiple elements into an existing list.
Syntax: list_name.extend(iterable)
Time Complexity: O(k), where k is the number of inserted elements.
- **insert() Method:** The insert() method inserts an element at a specified position.
Syntax: list_name.insert(index,value)
Time Complexity: O(n)

Problems

- Identify the even and odd numbers in the given list and separate them in 2 different list.
lst=list(range(101,201,1)).
- Identify the even and odd numbers in the given list and capture the differences in average of even_list and average of odd_list
lst=list(range(1,101,1)).

15.2.2 Deletion Operations

- **remove() Method:** The remove() method deletes the first matching value.
Syntax: list_name.remove(value)
Time Complexity: O(n)
- **pop() Method:** The pop() method removes an element from a list and returns the removed element.
Syntax: list_name.pop([index]). If no index is supplied , the last element is removed.
Time Complexity: O(1) or O(n) if indexed is given.

- **clear() Method:** The clear() method removes every element from the list.
Syntax: list_name.clear()
Time Complexity: O(n)

Problems

- Identify the count of all the data types in the given list.
lst=[1,2,3,5.3,3.2,2.1,True, False, 'abc', 'xyz']
- Perform element wise addition in 2 different list and output in a separate list.
l1=[1,2,'abc',3]
l2=[4,5,6,'def']

15.2.3 Miscellaneous Operations

- **count() Method:** Counts the number of occurrences of a specified value.
Syntax: list_name.count(value)
Time Complexity: O(n)
- **index() Method:** Returns the index of the first matching element.
Syntax: list_name.index(value)
Time Complexity: O(n)
- **sort() Method:** The sort() method arranges the elements of a list in ascending order by default.
Syntax: list_name.sort()
Time Complexity: O(nlogn)
- **reverse() Method:** Reverse the existing order of elements. It does not perform sorting.
Syntax: list_name.reverse()
Time Complexity: O(n)

Problems

- Fetch the index of second 'abc' (duplicate) in l1=[1,2,'abc',3].
- Count the occurrence of duplicates in the given list, lst=[1,2,3,2,3,1,4,5,6].

15.3 Dictionary(key-value pair)

- A dictionary is a mutable collection of key-value.
- Concept of indexes don't exist.
- Keys are unique in dictionary; Keys are only of immutable type: int,float,str,bool,tuple.
- Value can be anything

Syntax: d={key1:value1,key2:value2}

Problems

- Find common elements in the given tuples. Hint: Use Dictionary
t_x=(1,2,3,4,5)
t_y=(4,5,6,7)
- Count the occurrence of duplicates in the given list. Hint: Use Dictionary
lst=[1,2,3,2,3,1,4,5,6]
- Identify the count of all the data types in the given list. Hint: Use Dictionary
lst= [1,2,3,5.3,3.2,2.1,True,False,'abc','xyz']
- Find uncommon elements in the given tuples. Hint : Use Dictionary
t_x=(1,2,3,4,5)
t_y=(4,5,6,7)

16 Sets

Note: We avoid using sets due to following properties.

Properties:

1.Sorting the elements 2.Deleting the duplicates

`s1={1,2,3,4,3,2,1,4,5,6,7}` `s1`

Output:

`1,2,3,4,5,6,7`

The `dir()` function displays all available methods supported by the set object.

16.1 Set Operations

16.1.1 Union of Two Sets

The union operation returns every unique element that exists in either of the two sets.

Purpose: Combine two sets while automatically removing duplicates values.

Syntax: `set1.union(set2)`

16.1.2 Intersection of Two Sets

The intersection operation returns only the common elements.

Syntax: `s1.intersection(s2)`

16.1.3 Difference Operation

Difference returns elements that belong to one set but not the other. **Syntax:**`s1.difference(s2)`-Elements from `s1` except common elements

`s2.difference(s1)`-Elements from `s2` except common elements

16.1.4 Symmetric Difference

Symmetric difference returns elements that appear in exactly one of the two sets.

Syntax: `s1.symmetric_difference(s2)`

17 List Comprehension

A List Comprehension creates a new list by evaluating an expression for every element in an iterable. Instead of writing multiple lines using a for loop and `append()`, Python allows both operations to be written in a single statement.

Syntax: `new_list = [expression for item in iterable]`

syntax with condition: `new_list = [expression for item in iterable if condition]`

Problems

- Generate a list containing only even numbers between 1 and 100 by list comprehensive.
- Fetch Only even numbers after adding 100 to values<25 and 200 to values>=25. Hint: Use List Comprehensive

18 User-Defined Functions(UDF)

Python provides tow broad categories of functions:

– **Built-in (System-defined) Functions:**

* `sum()`

* `max()`

* `min()`

- * len()
- * print()
- * type()
- * abs ()
- * round()
- * sorted()

- **User-Defined Functions:** A User-Defined Function (UDF) is a function created by the programmer to perform a specific task. Function improve code reusability, readability and maintainability.

Syntax:

```
def function_name(parameters):
. statements
. return value
```

Problems

- * Write a UDF to add 2 elements. Perform proper data validation.
- * Write a UDF to perform arithmetic operations on 2 elements.

Python passes object references.

- Immutable Objects include
 - Integers
 - Floats
 - Strings
 - Tuples
- Mutable objects include
 - Lists
 - Dictionaries
 - Sets

19 Variable-Length Positional Arguments (*args)

Python allows a function to accept a variable number of positional arguments using the special syntax ***args-Stores all positional arguments inside a tuple.**

Instead of defining a fixed number of parameters, the function can receive zero, one ,or many positional arguments.

Problems

- Provide sum of n numbers with proper date validation. Hint: Use *args
- Write a UDF to perform element wise addition in 2 different list and output in a separate list.
 l1=[1,2,'abc',3,5]
 l2=[4,5,6,'def']
- Write a UDF to identify the missing number from a given set of n sequential numbers:
 1,2,3,4,6,7,8,9,10 - 5

20 Variable-Length Keyboard Arguments(**kwargs)

****kwargs** allows a function to accept any number of keyword arguments.

Unlike *args, which stores positional arguments inside a tuple, **kwargs stores keyword arguments inside a dictionary. ****kwargs is always a dictionary.**

21 Lambda Functions

A lambda function is an anonymous function that consists of a single expression. Unlike normal functions, it does not require the def keyword.

Syntax: lambda parameters: expression

Comparison

- Normal function
def square(x): . return x*x
- lambda x:x*x

22 Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm in which programs are organized around objects that combine data and behaviour.

23 Class

A class is a blueprint or template used to create objects.

A class can contain:

- Instance variables that store object data
- Methods that define object behaviour
- A constructor used to initialize objects

Syntax:

```
class ClassName:  
.. pass
```

Example: Employee

24 Object

An object is an instance of a class.

If a class is considered a blueprint, an object is a concrete entity created from that blueprint.

Examples: John and Ram

25 Constructor

The special method `__init__` is commonly used as the constructor for initializing an object.

Example: def `__init__(self, id, name, address, salary):` Constructor

26 Instance Variables

Instance variables are attributes associated with a particular object. **Example:** `self.id`- Store employee ID

27 Methods

A method is a function defined inside a class.

The class contains the following methods.

- `__init__`
- `get_id()`

- `set_id()`
- `get_name()`
- `set_name()`

28 Getter Methods

A getter method retrieves a value from an object.

29 Return Values

The method `get_id()` returns the employee ID.

The return statement sends the value back to the caller.

30 Setter Method

A setter method modifies an object's attribute. The setter receives a new value and stores it in the corresponding instance attribute. The setter methods do not explicitly use return, so they implicitly return None.

31 Creating an Object

Creates an object using:

```
emp1=Employee(123, 'abc', 'Delhi-NCR', 12345)
```

Here, `emp1` is an object of the `Employee` class.

32 OOP Principles

- **Encapsulation:** Encapsulation means combining related data and methods inside a class. The `Employee` class combines employee information with methods that retrieve and modify that information.
- **Abstraction:** Abstraction means exposing a useful interface while hiding implementation details. A user of the class can call:
`emp1.get_name()`
without needing to know how the method accesses `self.Name` internally.
- **Inheritance:** Inheritance allows a new class to reuse and extend another class. For example, specialized employee classes could inherit common employee functionality.
- **Polyorphism:** Polyorphism allows objects of different classes to provide different implementations of a common interface.
- **Method Resolution Order:** Method Resolution Order (MRO) determines the order in which Python searches classes for methods and attributes, particularly when inheritance is involved.
- **super():** The `super()` function is commonly used in inheritance to access functionality from a parent class.

33 Dunder Methods

Dunder methods are special methods whose names generally begin and end with double underscores.

- `__init__`
- `__str__`

- `__repr__`
- `__len__`
- `__getitem__`

34 `dir()`

The `dir()` function is an introspection tool that returns a list of names associated with an object, class, or module.

35 Installing a Python Library

`pip install LIBRARY_NAME`

Purpose: The command is used to install a Python package from the Python Package Index or another configured package repository.

35.1 `pip`

`pip` is the standard package-management tool commonly used for installing and managing Python packages.

35.2 `pip freeze`

The primary purpose of `pip freeze` is to obtain a list installed Python package and their versions.

35.3 Package:

A package can contain:

- Python modules
- Functions
- Classes
- Supporting files
- Dependencies required by other packages.

For Data Analysts, commonly used packages include:

- **pandas:** Read and manipulate datasets
- **numpy:** Numerical operations
- **matplotlib:** static charts
- **seaborn:** Statistical plots
- **openpyxl:** Workbook operations
- **scikit-learn:** Predictive modelling

36 NumPy

NumPy stands for Numerical Python. It is a Python library designed primarily for numerical computing, scientific computing, and efficient manipulation of multidimensional arrays.

The NumPy array using the following properties:

- **N-dimensional:** A NumPy ndarrays can have any number of dimensions.
- **Homogeneous Data:** NumPy arrays are generally homogeneous, meaning that all elements of an array have the same data type.
- **Vectorization:** Vectorization allows an operation to be performed on an entire array without explicitly writing a Python loop for every elements.
- **Broadcasting:** Broadcasting allows NumPy to perform operations between arrays or between an array and a scalar when their shapes are compatible.
Broadcasting is particularly useful when updating or calculating values across a portion of an array.

36.1 Importing NumPy

`import numpy as np` - To make the library available in current session.

- `import` instructs Python to load a module or library
- `numpy` identifies the library being imported
- `as` assigns an alias
- `np` becomes the alias used to access NumPy functionality

36.2 Creating a NumPy Array

```
n1=np.array([1,2,3])  
type(a1)
```

Purpose: The purpose of this code is to convert a Python sequence into a NumPy ndarray.

- `np.array([1,2,3])` creates a NumPy array
- The resulting array is assigned to `a1`
- `type(a1)` checks the Python type of the result object.

36.3 Array Properties and Attributes

- **shape:** `a1.shape`
The shape attribute returns the dimensions of the array.
shape describes the size of the array along each dimension.
- **ndim:** `a1.ndim`
The ndim attribute returns the number of dimensions of the array.
- **size:** `a1.size`
The size attribute returns the total number of elements in the array.
- **dtype:** `a1.dtype`
The dtype attribute identifies the data type used to store the elements of the array.
- **nbytes:** `a1.nbytes`
The nbytes attribute returns the total number of bytes occupied by the array elements.
`nbytes = size * bytes per element`

36.4 Controlling the Data Type

```
a2= np.array([1,2,3], dtype=np.int8)
```

Here, `dtype` explicitly specifies the data type.

36.5 Creating Arrays

- **np.zeros():** np.zeros(shape = (4,4))

Purpose: np.zeros() creates an array whose elements are initialized to zero. The argument shape=(4,4) specifies four rows and four columns.

Syntax: np.zeros(shape), where the shape parameter specifies the required dimensions.

- **np.ones():** np.ones() creates an array where every element is initialized to one.
 - **np.zeros_like():** np.zeros_like() creates an array containing zeros with the same shape as the reference array. It is useful when a new array must have the same structural characteristics as an existing array.
 - **np.ones_like():** np.ones_like() creates an array containing one using the structure of the reference array.
 - **np.full():** np.full(shape=(3,3),fill_value='abc')
- np.full() creates an array with the specified shape and fills every element with the supplied value. Here,
- shape=(3,3) specifies three rows and three columns
 - fill_value='abc' specifies the value assigned to every element.
- **np.full_like():** np.full_like() creates a new array based on the shape and characteristics of an existing array while assigning the specified fill value.
 - **np.identity():** np.identity() creates a square identity matrix.

36.6 Creating Sequences with np.arange()

np.arange(1,101,0.5)

The parameters are:

- start: starting value
- end: stopping boundary
- step: difference between consecutive values

36.7 Creating Evenly Spaced Values with np.linspace()

np.linspace(1,100,5)

np.linspace() generates a specified number of evenly spaced values between a starting point and an ending point.

The arguments here are conceptually:

- Starting value : 1
- Ending value : 100
- Number of values : 5

36.8 Generating Random Values with np.random.random()

np.random.random(size = (3,3))

np.random.random() generates random floating-point values.

The argument: size = (3,3) specifies a two-dimensional array with three rows and three columns.

36.9 Generating Random Integers with np.random.randint()

```
a3 = np.random.randint(low = 1, high = 10, size = (4,5), dtype=np.int16)
```

Purpose: This statement creates a two-dimensional NumPy array containing randomly generated integers.

Parameters:

- low=1: lower boundary
- high=10: upper boundary
- size=(4,5): required shape
- dtype=np.int16: requested integer data type

36.10 Transposing an Array

```
np.transpose(a3)
```

Transposition changes the arrangement of dimension by exchanging rows and columns for a two-dimensional array.

36.11 Flattening an Array

```
a3.flatten()
```

flatten() converts the array into a one-dimensional copy.

36.12 NumPy Array Slicing and Dicing

Array slicing allows selected portions of an array to be extracted without processing every element individually.

The general slicing structure is:

```
array[start:end:step]
```

- start: starting index
- end: stopping boundary
- step: number of positions to move at each step

For a two-dimensional array, slicing can be applied independently to rows and columns:

```
array[row_start:row_end:row_step, column_start: column_end:column_step]
```

Problems

- Fetch the even rows from `a3 = np.random.randint(low = 1, high = 10, size = (4,5), dtype=np.int16)` (odd indexes)
- Fetch odd rows from `a3`
- Fetch data in odd rows and even columns from `a3`
- Fetch data in last row from `a3`
- Fetch data in last column from `a3`

36.13 Reshaping NumPy Arrays

```
a3.reshape((2,10))
```

The purpose of reshape() is to change the structural dimensions of an array while preserving the number of elements.

36.14 Boolean Filtering

Boolean filtering allows NumPy arrays to be filtered using Boolean conditions.

`array[condition]`

Bit wise operators - and by `&`; or by `|`; not by `~`

Problems

- Fetch only positive number from `a5=np.random.randint(-10,10, size=(4,5))`
- Fetch only even numbers from `a5`
- Fetch the positive even numbers from `a5`.
- Fetch the positive odd and negative even numbers from `a5`
- Replace the 9,5,4 with the twice their value from `a3`

36.15 Conditional Selection with `np.where()`

`np.where()` performs conditional selection.

Its basic conceptual syntax is:

`np.where(condition, value_if_true, value_if_false)`

The three important components are:

- condition
- Value returned when the condition is true
- Value returned when the condition is false

Problems

- Assign 'high sales' to values ≥ 100 otherwise 'low sales'
- Assign 'high sales' to values ≥ 100 otherwise 'low sales' if values ≥ 10 and 'poor sales' to values ≤ 10

37 Pandas

Pandas is a Python library designed primarily for working with structured and tabular data.

In practical Data Analyst work, Pandas is commonly used for importing, inspecting, transforming, filtering, summarizing and preparing datasets.

The two central data structures introduced in the notebook are:

37.1 Series:

A Series is a one-dimensional labeled data structure in Pandas.

Series contains:

- Data Values
- An index associated with each value
- A data type

37.1.1 Properties of a Series

- **One-dimensional:** A Series has one principal axis containing its indexed values.
- **Homogeneous:** A Series is generally associated with one primary data type.
- **Vectorization:** Vectorization means performing an operation on an entire collection of values rather than explicitly processing each value one at a time.
- **Broadcasting:** Broadcasting allows compatible scalar or array-like operations to be applied across multiple values.
For example, an operation involving a scalar and a Series can be applied to the Series values collectively.

37.1.2 Series Attributes

- **ndim:** Number of dimensions
- **dtype:** Data type
- **nbytes:** Memory used by data
- **shape:** Dimensions of structure
- **size:** Number of elements

```
s1= pd.Series([2,1,3,4])
s2= pd.Series([2,1,3,4], dtype = np.int32)
s3= pd.Series([2,1,4,3,5], index = ['a', 'b', 'c', 'd', 'e'])
s4= pd.Series(['$15', '$20', '$10'])
s5=[] and s5.append(int(item.replace('$', '')))
s6= pd.Series([3,2,1], index = ['a','b','c'], name='increment')
```

37.1.3 loc Indexer:

The loc indexer is primarily label-oriented. E.g: s3.loc['a','c']

37.1.4 iloc Indexer:

The iloc indexer is primarily position-oriented. E.g: s3.iloc[1:3]

Problems

- Fetch only even elements from s3
- Fetch the elements stored at even indexes in s3
- Find the sum of s4 element

Accessing Series Indexes: Series.index

Accessing Series Values: Series.values

37.2 DataFrame:

A DataFrame is the primary two-dimensional tabular data structure in Pandas.

A DataFrame can contain multiple columns with potentially different data types.

Creating a DataFrame from a Dictionary:

```
d1= pd.DataFrame({'col1': [100,200,300], 'col2':['abc','xyz','lmn'], 'col3':[True, False, True]})
```

37.2.1 DataFrame Attributes:

- **DataFrame.shape[]:** The shape attribute returns a tuple: (number of rows, number of columns)
- **DataFrame.ndim:** Number of Dimensions
- **DataFrame.size:** Number of Elements
- **DataFrame.dtypes:** Data Types
- **DataFrame.loc[i]:** Extract only the i-1 row
- **DataFrame.iloc[:, -1]:** Extract the last column
- **DataFrame.shape[1]:** It returns the number of columns

A Series can be viewed as a one-dimensional labeled data structure, whereas a DataFrame can be viewed as a two-dimensional table consisting of rows and columns.

38 Data Analysis Workflow

In data analysis (pandas), we usually do followings steps:

38.1 Requirement gathering

38.2 Data Import

38.3 Understanding the data

- Metadata auditing (Metadata- data about data)
- Data Auditing

38.4 Data prep—Data Cleaning—Data wrangling—Data munging

- Structure based changes
 - Subsetting
 - Reordering the columns
 - Change the column names
 - Data type conversion
- Data based changes
 - Handling the duplicates
 - Missing value imputation
 - Handling the outliers
 - Sorting + Filtering
 - Summaries of data
 - Consolidating the data (combining the data)
 - * Data Append (horizontal/verical) - data concatenation
 - * Merge/Join (outer, inner, left, right, self, cross)
 - Groups & Bins
 - Functions
- BI : Build charts/graphs

39 Data Preparation

39.1 Structure-based changes

- **Subsetting:** Subsetting means selecting a limited portion of a dataset.
- **Reordering columns:** Reordering columns means changing the order in which variables appear in the DataFrame.
- **Changing Column Names**
- **Data Type Conversion:** Data type conversion means changing how a column's values are represented or interpreted.
Data types are important because Pandas performs different operations depending on whether data is numerical, textual, Boolean, or another supported type.

39.2 Data-Based Changes

- **Handling duplicates:** A duplicate occurs when records or relevant combinations of fields repeat in a dataset.
- **Missing-value imputation:** Missing values occur when information is unavailable, not recorded, or otherwise absent.
- **Handling Outliers:** An outlier is an observation that is unusually distant from the general pattern of the data.
- **Sorting and Filtering:** Filtering selects observations satisfying a condition.
Sorting changes the ordering of observations according to a selected variable. Sorting can make patterns easier to inspect and can support reporting requirements.
- **Summaries of Data**
- **Data Consolidation:** Data Consolidation is the process of combining data. Data consolidation is important when information is distributed across multiple datasets.
- **Merge and Join:**
 - **Outer Join:** An outer join preserves records from both sides, including records that do not have a corresponding match.
 - **Inner Join:** An inner join retains records satisfying the matching relationship between the participating datasets.
 - **Left Join:** A left join preserves the records from the left-side dataset while matching available information from the other dataset.
 - **Right Join:** A right join preserves the records from the right-side dataset while matching available information from the other dataset.
 - **Self Join:** A self join combines a dataset with itself. It is useful when relationships exist between records within the same dataset.
 - **Cross Join:** A cross join produces combinations between records of the participating datasets. The resulting number of combinations can become very large.
- **Groups and Bins:** Grouping organizes observation according to a selected category or classification. Groups allow an analyst to summarize data at a meaningful business level.
Binning divides continuous values into predefined ranges or categories. Bins transform continuous ranges into categories.
- **Functions:** Functions allow reusable analytical logic to be represented in a structured form. A function can:
 - Receive input parameters
 - Perform an operation

- Return a result
- Encapsulate reusable logic

Functions provide a mechanism for organizing operations and separating logic into reusable units.

- **Business Intelligence: Charts and Graphs:** Charts and graphs convert prepared data into visual information.